第 1 章 基础算法（下）

8. 贪心

贪心算法是**两极分化很严重的算法**。简单的问题会让你觉得理所应当，难一点的问题会让你怀疑人生。

1. 什么是贪心算法？

> 贪心算法，或者说是贪心策略：企图用局部最优找出全局最优。
>
> 1. 把解决问题的过程分成若干步；
> 2. 解决每一步时，都选择"当前看起来最优的"解法；
> 3. "希望"得到全局的最优解。

2. 贪心算法的特点

> 1. 对于大多数题目，贪心策略的提出并不是很难，难的是证明它是正确的。因为贪心算法相较于暴力枚举，每一步并不是把所有情况都考虑进去，而是只考虑当前看起来最优的情况。但是，局部最优并不等于全局最优，所以我们必须要能严谨的证明我们的贪心策略是正确的。
>
>   一般证明策略有：反证法，数学归纳法，交换论证法等等。
> 2. 当问题的场景不同时，贪心的策略也会不同。因此，贪心策略的提出是没有固定的套路和模板的。我们后面讲的题目虽然分类，但是大家会发现具体的策略还是相差很大。
>
>   因此，不要妄想做几道贪心题目就能遇到一个会一个。有可能做完 50 道贪心题目之后，第 51 道还是没有任何思路。

3. 如何学习贪心？

> 先有一个认知：做了几十道贪心的题目，遇到一个新的又没有思路，这时很正常的现象，把心态放平。
>
> 1. 前期学习的时候，重点放在各种各样的策略上，把各种策略当成经验来吸收；
> 2. 在平常学习的时候，我们尽可能的证明一下这个贪心策略是否正确，这样有利于培养我们严谨的思维。但是在比赛中，能想出来一个策略就已经不错了，如果再花费大量的时间去证明，有点得不偿失。这个时候，如果根据贪心策略想出来的若干个边界情况都能过的话，就可以尝试去写代码了。

8.1 简单贪心

8.1.1 货仓选址

题目来源： 洛谷

题目链接： P10452 货仓选址

难度系数： ★

【题目描述】

在一条数轴上有 n 家商店，它们的坐标分别为 $a_1, a_2, ..., a_n$。

现在需要在数轴上建立一家货仓，每天清晨，从货仓到每家商店都要运送一车商品。

为了提高效率，求把货仓建在何处，可以使得货仓到每家商店的距离之和最小。

【输入描述】

第一行一个整数 n，第二行 n 个整数 $a_1, a_2, ..., a_n$

对于 100% 的数据: $N \leq 100000, a_i \leq 1000000$

【输出描述】

一个整数，表示距离之和的最小值。

【示例一】

输入：

```
4
6 2 9 1
```

输出：

```
12
```

【解法】

将所有的商店按照「从小到大」的顺序「排序」，把货仓建在中位数处，可以使得货仓到每家商店的距离之和最小：

- 如果 n 是奇数，货仓建在 $a[(n+1)/2]$ 位置处；
- 如果 n 是偶数，货仓建在 $a[n/2] \sim a[n/2+1]$ 之间都是可以的

拓展：从这道题中我们可以得出一个结论（往后做题的时候可能会用到）：

形如：$sum = \sum_{i=1}^{n} |a[i] - x| = |a[1] - x| + |a[2] - x| + ... + |a[n] - x|$ 这样一个式子：

- 当 x 取到 n 个数的中位数时，和最小；
- 最小和为：$(a[n] - a[1]) + (a[n-1] - a[2]) + ... + (a[n+1-n/2] - a[n/2])$。

【参考代码】

代码块

```cpp
#include <iostream>
#include <algorithm>

using namespace std;

typedef long long LL;

const int N = 1e5 + 10;

int n;
LL a[N];

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];
    sort(a + 1, a + 1 + n);

    LL ret = 0;
    // 利用中间值来计算
    // for(int i = 1; i <= n; i++)
    // {
    //     ret += abs(a[i] - a[n / 2]);
    // }

    // 用结论计算
    for(int i = 1; i <= n / 2; i++)
    {
        ret += a[n - i + 1] - a[i];
    }

    cout << ret << endl;

    return 0;
}
```

8.1.2 最大子段和

> **题目来源：** 洛谷
>
> **题目链接：** P1115 最大子段和

> **难度系数：** ★★

【题目描述】

> 给出一个长度为 n 的序列 a ，选出其中连续且非空的一段使得这段和最大。

【输入描述】

> 第一行是一个整数，表示序列的长度 n 。
>
> 第二行有 n 个整数，第 i 个整数表示序列的第 i 个数字 a_i 。
>
> - 对于 40% 的数据，保证 $n \leq 1 \times 10^3$ 。
> - 对于 100% 的数据，保证 $1 \leq n \leq 2 \times 10^5, -10^4 \leq a_i \leq 10^4$ 。

【输出描述】

> 输出一行一个整数表示答案。

【示例一】

> 输入：
>
> 7
>
> 2 -4 3 -1 2 -4 3
>
> 输出：
>
> 4

【解法】

有没有似曾相识的感觉？这是我们第二次遇见它了，但还不是最后一次~

贪心想法：从前往后累加，我们会遇到下面两种情况：

- 目前的累加和 ≥ 0 ：那么当前累加和还会对后续的累加和做出贡献，那我们就继续向后累加，然后更新结果；
- 目前的累加和 < 0 ：对后续的累加和做不了一点贡献，直接大胆舍弃计算过的这一段，把累加和重置为 0 ，然后继续向后累加。

这样我们在扫描整个数组一遍之后，就能更新出最大子段和。

如果是善于思考的同学此时就会有「一些」大大的疑惑了， $why?\ why?\ why?$ 为什么可以得到「最优解」？怎么感觉这个策略是「错」的啊？感觉「好多情况」都没考虑进去，为什么就得到一个正确的结果？为什么可以「大胆舍去」这一段累加和？

如果你有大大的疑惑，这就对了。我刚开始做这道题，看到别人的题解是这样写的时候，也有如此疑惑。（我觉得这就是贪心算法的魅力吧，看似很简单，很玄学，其实有很多值的我们思考的地方）别

着急，我们接下来证明一下这个贪心策略是正确的。

其实只需要证明我们在累加的过程中，出现负数时，为什么可以大胆的舍去这一段区间，然后重新开始。证明以下三点，就可以「大胆舍弃」了：

在累加的过程中算出一段区间和 $sum[a,b]<0$，如果不舍弃这一段，那么 $[a,b]$ 段之间就会存在一点，「以某个位置为起点」就会「更优」，分为下面两种情况：

1. 在 ab 段存在一个点 c，从这个位置开始，「越过 b 」的累加和比从 a 开始的累加和更优：

> 用「反证法」证明这种情况不存在。
>
> 如果存在这一点，那么：$sum[c,b]>sum[a,b]$，这样才能保证向后加的时候更优。
>
> 但这是「不可能」的。如果 $sum[c,b]>sum[a,b]$，那么 $sum[a,c-1]<0$，这与我们的贪心策略矛盾。
>
> 因为我们贪心策略向后加的时候，只要不小于 0，就会一直加下去。如果 $[a,c-1]$ 段小于 0，就会在 c 点之前停止，不会累加到 b。
>
> 因此区间内不存在一点，在计算子数组和时，在「越过 b 」的情况下，能比从 a 开始更优。

2. 在 ab 段存在一个点 c，从这个位置开始，「不越过 b 」的累加和比从 a 开始的累加和更优：

> 也可以用「反证法」证明这种情况不存在。
>
> 如果存在这一点，那么：$sum[c,k]>sum[a,k]$。
>
> 但这是不可能的。如果 $sum[c,k]>sum[a,k]$，那么 $sum[a,c-1]<0$，这与我们的贪心策略矛盾。
>
> 因此区间内不存在一点，在计算子数组和时，在「不越过 b 」的情况下，能比从 a 开始更优。

综上所述，我们可以大胆舍弃这一段，重新开始。

【参考代码】

代码块

```
#include <iostream>

using namespace std;

typedef long long LL;
```

```cpp

const int N = 2e5 + 10;

int n;
LL a[N];

int main()
{
    cin >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];

    LL sum = 0, ret = -1e6;
    for(int i = 1; i <= n; i++)
    {
        sum += a[i];
        ret = max(ret, sum);
        if(sum < 0) sum = 0;
    }

    cout << ret << endl;

    return 0;
}
```

8.1.3 纪念品分组

题目来源： 洛谷

题目链接： P1094 [NOIP2007 普及组] 纪念品分组

难度系数： ★

【题目描述】

元旦快到了，校学生会让乐乐负责新年晚会的纪念品发放工作。为使得参加晚会的同学所获得 的纪念品价值相对均衡，他要把购来的纪念品根据价格进行分组，但每组最多只能包括两件纪念品， 并且每组纪念品的价格之和不能超过一个给定的整数。为了保证在尽量短的时间内发完所有纪念品，乐乐希望分组的数目最少。

你的任务是写一个程序，找出所有分组方案中分组数最少的一种，输出最少的分组数目。

【输入描述】

共 $n+2$ 行：

第一行包括一个整数 w，为每组纪念品价格之和的上限。

第二行为一个整数 n，表示购来的纪念品的总件数 G。

> 第 $3 \sim n+2$ 行每行包含一个正整数 p_i 表示所对应纪念品的价格。
>
> 100% 的数据满足： $1 \leq n \leq 3 \times 10^4, 80 \leq w \leq 200, 5 \leq p_i \leq w$ 。

【输出描述】

> 一个整数，即最少的分组数目。

【示例一】

> 输入：
>
> 100
> 9
> 90
> 20
> 20
> 30
> 50
> 60
> 70
> 80
> 90
>
> 输出：
>
> 6

【解法】

先将所有的纪念品排序，每次拿出当前的最小值 x 与最大值 y ：

- 如果 $x+y \leq w$ ：就把这两个放在一起；
- 如果 $x+y>w$ ：说明此时最大的和谁都凑不到一起， y 单独分组， x 继续留下在进行下一次判断。

直到所有的物品都按照上述规则分配之后，得到的组数就是最优解。

可以用「交换论证法」证明贪心解就是最优解：

> 对于区间 $[a_i \ldots\ldots a_j]$ ，如果存在最优解，但是 a_i 与 a_j 的分配方式与我们贪心解的分配方式不一样，那么就会有以下几种情况：
>
> 1. 当 $a[i]+a[j]>w$ 时：

- 贪心解会把 $a[j]$ 单独分组， $a[i]$ 留待下次考虑；
- 最优解也必定会把 $a[j]$ 单独分组，因为没有更小的值与 $a[j]$ 组合。

此时贪心解与最优解一致。

2. 当 $a[i]+a[j]\le w$ 时：

- 贪心解会把两者组合分在一个组里面；
- 最优解可能有以下几种情况：
  - $a[j]$ 单独一组：
    - 如果 $a[i]$ 也单独一组的话，最优解还不如贪心解分的组少，矛盾；
    - 如果 $a[i]$ 和另一个 $a[k]$ 一组的话，我们可以把 $a[k]$ 与 $a[j]$ 交换，此时并不影响结果，和贪心解一致。
  - $a[j]$ 和 $a[k]$ 一组：
    - 如果 $a[i]$ 单独一组的话，交换 $a[i]$ 和 $a[k]$，此时并不影响最终结果，和贪心解一致；
    - 如果 $a[i]$ 和 $a[l]$ 一组的话，交换 $a[i]$ 和 $a[k]$，此时变成 $(a[i]+a[j]),(a[l]+a[k])$，其中 $a[l]+a[k]\le a[j]+a[k]\le w$，不影响最终结果，和贪心解一致。

综上所述，我们可以通过不断的「调整」，使的最优解在「不改变其最优性」的前提下，变得和贪心解一致。那我们的贪心策略就等价于最优策略。

【参考代码】

代码块

```
#include <iostream>
#include <algorithm>

using namespace std;

const int N = 3e4 + 10;

int w, n;
int a[N];

int main()
{
    cin >> w >> n;
    for(int i = 1; i <= n; i++) cin >> a[i];
    sort(a + 1, a + 1 + n);

    int l = 1, r = n, ret = 0;
```

```
18      while(l <= r)
19      {
20          if(a[l] + a[r] <= w) l++, r--;
21          else r--;
22          ret++;
23      }
24      cout << ret << endl;
25
26      return 0;
27  }
```

8.1.4 排座椅

题目来源： 洛谷

题目链接： P1056 [NOIP2008 普及组] 排座椅

难度系数： ★★

【题目描述】

上课的时候总有一些同学和前后左右的人交头接耳，这是令小学班主任十分头疼的一件事情。不过，班主任小雪发现了一些有趣的现象，当同学们的座次确定下来之后，只有有限的D对同学上课时会交头接耳。

同学们在教室中坐成了 M 行 N 列，坐在第 i 行第 j 列的同学的位置是 (i,j)，为了方便同学们进出，在教室中设置了 K 条横向的通道， L 条纵向的通道。

于是，聪明的小雪想到了一个办法，或许可以减少上课时学生交头接耳的问题：她打算重新摆放桌椅，改变同学们桌椅间通道的位置，因为如果一条通道隔开了两个会交头接耳的同学，那么他们就不会交头接耳了。

请你帮忙给小雪编写一个程序，给出最好的通道划分方案。在该方案下，上课时交头接耳的学生对数最少。

【输入描述】

第一行，有 5 各用空格隔开的整数，分别是
$M,N,K,L,D(2 \le N,M \le 1000, 0 \le K < M, 0 \le L < N, D \le 2000)$

接下来 D 行，每行有 4 个用空格隔开的整数，第 i 行的 4 个整数 X_i,Y_i,P_i,Q_i，表示坐在位置 (X_i,Y_i) 与 (P_i,Q_i) 的两个同学会交头接耳（输入保证他们前后相邻或者左右相邻）。

输入数据保证最优方案的唯一性。

【输出描述】

共两行

第一行包含 K 个整数， $a_1, a_2, ..., a_K$ ，表示第 a_1 行和 a_1+1 行之间、第 a_2 行和第 a_2+1 行之间、…、第 a_K 行和第 a_K+1 行之间要开辟通道，其中 $a_i < a_{i+1}$ ，每两个整数之间用空格隔开（行尾没有空格）。

第二行包含 L 个整数， $b_1, b_2, ..., b_L$ ，表示第 b_1 行和 b_1+1 行之间、第 b_2 行和第 b_2+1 行之间、…、第 b_K 行和第 b_L+1 行之间要开辟通道，其中 $b_i < b_{i+1}$ ，每两个整数之间用空格隔开（行尾没有空格）。

【示例一】

输入：

```
4 5 1 2 3
4 2 4 3
2 3 3 3
2 5 2 4
```

输出：

```
2
2 4
```

【解法】

由题意可得，我们会发现一些性质：

- 设置横向通道的时候，并「不影响」左右相邻的同学；
- 设置纵向通道的时候，并「不影响」上下相邻的同学。

因此我们可以「分开」处理横向通道和纵向通道。

处理横向通道（纵向同理，就不多赘述）：

- 收集每一行如果放上通道之后，会解决多少个交头接耳的同学；
- 对收集的信息「从大到小」排序，选最大的 k 行就是最优结果。

不需要证明了吧……我们都把最大的 k 个拿出来了，如果还不是最优的话找谁说理去。

【参考代码】

代码块

```cpp
#include <iostream>
#include <algorithm>

```

```cpp
using namespace std;

const int N = 1010;

struct node
{
    int index;
    int cnt;
}row[N], col[N];

int m, n, k, l, d;

// 按照 cnt 从大到小排序
bool cmp1(node& x, node& y)
{
    return x.cnt > y.cnt;
}

// 按照 index 从小到大排序
bool cmp2(node& x, node& y)
{
    return x.index < y.index;
}

int main()
{
    cin >> m >> n >> k >> l >> d;
    // 初始化结构体数组
    for(int i = 1; i <= m; i++) row[i].index = i;
    for(int i = 1; i <= n; i++) col[i].index = i;

    while(d--)
    {
        int x, y, p, q; cin >> x >> y >> p >> q;
        if(x == p) col[min(y, q)].cnt++;
        else row[min(x, p)].cnt++;
    }

    // 对两个数组按照 cnt 从大到小排序
    sort(row + 1, row + 1 + m, cmp1);
    sort(col + 1, col + 1 + n, cmp1);

    // 对 row 数组，前 k 个元素，按照下标从小到大排序
    sort(row + 1, row + 1 + k, cmp2);
    // 对 col 数组，前 l 个元素，按照下标从小到大排序
    sort(col + 1, col + 1 + l, cmp2);

```

```
51        for(int i = 1; i <= k; i++)
52        {
53            cout << row[i].index << " ";
54        }
55        cout << endl;
56        for(int i = 1; i <= l; i++)
57        {
58            cout << col[i].index << " ";
59        }
60        cout << endl;
61
62        return 0;
63    }
```

8.1.5 矩阵消除游戏

> **题目来源：** 牛客网
>
> **题目链接：** 矩阵消除游戏
>
> **难度系数：** ★★★

【题目描述】

> 牛妹在玩一个名为矩阵消除的游戏，矩阵的大小是 n 行 m 列，第 i 行第 j 列的单元格的权值为 $a_{i,j}$ ，牛妹可以进行 k 个回合的游戏，在每个回合，牛妹可以选择一行或者选择一列，然后将这一行或者这一列的所有单元格中的权值变为 0 ，同时牛妹的分数会加上这一行或者这一列中的所有单元格的权值的和。
>
> 牛妹想最大化她的得分，球球你帮帮她吧！

【输入描述】

> 第一行三个整数 n, m, k
> 接下来 n 行每行 m 个整数表示矩阵中各个单元格的权值
>
> $1 \leq n, m \leq 15$
>
> $1 \leq a_{i,j} \leq 10^6$
>
> $1 \leq k \leq n * m$

【输出描述】

> 输出一个整数表示牛妹能获得的最大分数。

【示例一】

> 输入：

3 3 2

101 1 102

1 202 1

100 8 100

输出：

414

【解法】

错误的贪心：每次都选当前看起来最好的一行或者一列，然后选完之后，修改原数组，继续重复刚刚的操作。

> 反例，当 $k = 2$ 时：
>
> ```
> 100  9   10
> 0    0   0
> 100  0   10
> ```
>
> 如果是贪心的话，会选第一列和最后一列。但是最优情况下我们可以把所有的数都选上，因此直接上来就贪心不是最优解。

直接贪心的错误在于，我们每次选完「一行」或者「一列」之后，会对接下来的「列」或者「行」的选择造成影响。比如上面的反例：

- 我们先贪心的选了第一列。站在「行」的视角上，在「列」没选之前，我分明能贪心的选「第一行」，但是你把「第一列」拿走哦之后，就会对「第一行」造成影响。

既然行的选择会影响列，那我们干脆直接把「所有行的选法」枚举出来，然后针对「每一种行的选法」再处理列，这样就会把「所有情况」都考虑进去。

因此，最优解是先暴力枚举所有行的选法，在行的选择都确定之后，再去贪心的处理列。这个思想我们是不陌生的，参考二进制枚举里面《费解的开关》。

【参考代码】

代码块

```
#include <iostream>
#include <algorithm>
#include <cstring>

using namespace std;

const int N = 20;
```

```cpp

int n, m, k;
int a[N][N];
int col[N]; // 统计列和

// 统计 x 的二进制表示中 1 的个数
int calc(int x)
{
    int ret = 0;
    while(x)
    {
        ret++;
        x -= x & -x;
    }
    return ret;
}

// 按照值从大到小排序
bool cmp(int a, int b)
{
    return a > b;
}

int main()
{
    cin >> n >> m >> k;
    for(int i = 0; i < n; i++)
        for(int j = 0; j < m; j++)
            cin >> a[i][j];

    int ret = 0;
    // 暴力枚举出行的所有选法
    for(int st = 0; st < (1 << n); st++)
    {
        int cnt = calc(st);
        if(cnt > k) continue; // 不合法的状态

        memset(col, 0, sizeof col);
        int sum = 0; // 记录当前选法中的和
        for(int i = 0; i < n; i++)
        {
            for(int j = 0; j < m; j++)
            {
                if((st >> i) & 1) sum += a[i][j];
                else col[j] += a[i][j];
            }
        }
```

```
55
56          // 处理列
57          sort(col, col + m, cmp);
58          // 选 k - cnt 列
59          for(int i = 0; i < k - cnt; i++) sum += col[i];
60          ret = max(ret, sum);
61      }
62
63      cout << ret << endl;
64
65      return 0;
66  }
```

8.2 推公式

如果细说的话，这个专题应该叫**推公式+排序**。其中推公式就是**寻找排序规则**，排序就是在该排序规则下**对整个对象排序**。

在解决某些问题的时，当我们发现**最终结果需要调整每个对象的先后顺序**，也就是对整个对象排序时，那么我们就可以用推公式的方式，得出我们的排序规则，进而对整个对象排序。

正确性证明：

> 利用排序解决问题，最重要的就是需要证明"**在新的排序规则下，整个集合可以排序**"。这需要用到**离散数学中"全序关系"的知识**。我会在第一道题中证明该题的排序规则下，整个集合是可以排序的。
>
> 但是证明过程很麻烦，后续题目中我们**只要发现该题最终结果需要排序，并且交换相邻两个元素的时候，对其余元素不会产生影响**，那么我们就可以推导出排序的规则，然后直接去排序，就不去证明了。

8.2.1 拼数

> **题目来源：** 洛谷
>
> **题目链接：** P1012 [NOIP1998 提高组] 拼数
>
> **难度系数：** ★★

【题目描述】

> 设有 n 个正整数 $a_1, a_2, ..., a_n$ ，将它们联接成一排，相邻数字首尾相接，组成一个最大的整数。

【输入描述】

> 第一行有一个整数，表示数字个数 n 。
>
> 第二行有 n 个整数，表示给出的 n 个整数 a_i 。